

## 40005 Advanced iOS Development: Assessment 3

# SkillSwap

## Platform-Integrated iOS Solution

40005 - Software Development

Student Name      Zade Elsaddik

Student ID        26077394

### Required Document

Problem Statement | Design Justification | Architecture Diagram | Reflective Report

# Document Contents

**Section 1:** Problem Statement

**Section 2:** Design Justification

**Section 3:** Architecture Diagram

**Section 4:** Reflective Report

**References:** Sources used in the problem justification

## Section 1 - Problem Statement

### The problem

SkillSwap is designed around a fairly common problem for university students and young adults: they sometimes need practical help, tutoring, creative support, technology assistance or another small service, but paying for a professional is not always realistic. At the same time, the person asking for help may already have a useful skill of their own that another person would value. The problem is not that these skills do not exist. The problem is that there is usually no simple, structured way to match what one person needs with what they can offer in return.

This becomes more relevant when cost-of-living pressure is considered. An Australian Department of Education review of student equity found that rising living costs were repeatedly identified as a barrier to successful study. It noted that some students were working substantial hours simply to cover necessities such as transport, textbooks and everyday expenses, which can reduce the time available for study. The same review described financial strain as an ongoing concern for students with fewer resources to draw on (Tomaszewski et al., 2023). SkillSwap does not solve the wider cost-of-living problem, but it is designed to reduce one small source of friction by giving people another way to access minor help without every interaction needing to become a paid transaction.

There is also evidence that reciprocal service-exchange systems are a real model rather than only a hypothetical idea. Research into timebanking describes platforms where members post requests and offers, exchange services and draw on skills that already exist within a community. Yuan, Hanrahan and Carroll (2018) found that service-exchange tools can help connect people, pool community resources and support trust and reciprocity. SkillSwap uses a simpler direct exchange model, but the basic idea is similar: people are treated as both someone who may need help and someone who has something useful to contribute.

### Primary stakeholder

**Primary stakeholder:** a university student or young adult with limited disposable income who occasionally needs practical help and has a skill they can offer in return.

The stakeholder is deliberately specific. SkillSwap is not trying to be a full employment platform, a professional-services marketplace or a general social network. It focuses on relatively small exchanges such as tutoring for design help, language practice for technology assistance, basic repairs for photography help, or similar situations where both people can contribute something useful.

### What happens without a structured solution

Without a dedicated system, these exchanges are usually handled informally through group chats, social media posts, friends or word of mouth. That makes it easy for important details to become scattered: what help is

actually needed, what is being offered in return, whether the request is still open, who has made an offer, and whether an exchange has already been matched.

The human cost is mostly friction rather than a single dramatic failure. A student may give up on getting minor help because paying is not worthwhile, spend extra time searching across different channels, or agree to an exchange without a clear record of what both sides expected. SkillSwap tries to make that process more visible and organised by treating the request, the offered skill, incoming offers and the final match as explicit parts of the domain.

## Proposed solution

SkillSwap lets a member post a two-sided skill request: what they need and what they can offer in return. Other members can browse open requests, make an offer, and the request owner can accept one offer or close the request. The app keeps the state of the exchange clear, stores the data locally, and uses iOS extensions so relevant information can appear outside the main app. The intention is to support a small, reciprocal exchange from discovery through to a confirmed match without turning it into a conventional paid marketplace.

## Section 2 - Design Justification

### Why an iOS app?

A mobile app suits this problem because the interaction is lightweight and often happens away from a desk. A person may think of a request while travelling, notice an open exchange while between classes, or want to check an offer quickly. SwiftUI provides a straightforward way to build the main interaction around short, task-focused screens such as Home, Explore, Post a Swap, Activity, Profile and Request Detail.

iOS also gives SkillSwap useful system-level capabilities that a basic web page would not provide as naturally. WidgetKit can surface exchange information on the Home Screen, local notifications can remind a user about a matched exchange, and an Action Extension can accept text from another app. These features support the actual workflow instead of requiring the user to remember to reopen SkillSwap every time.

### Why these extensions?

#### WidgetKit Widget Extension

The widget is useful for a user who wants a quick view of SkillSwap activity without opening the full application. It displays a compact snapshot of relevant exchange information and supports small and medium widget families. The main app publishes a lightweight snapshot to the App Group and reloads the widget after relevant request or offer changes. This keeps the extension focused on glanceable information rather than trying to make the widget a second copy of the application.

#### Notification Content Extension

Once an exchange has been matched, the reminder is no longer generic. The user needs to know who the exchange is with and what each side of the swap involves. The Notification Content Extension handles the SKILLSWAP\_SESSION\_REMINDER category and presents partner and skill information in domain language. This makes the notification useful on its own instead of only saying that 'something is due'.

#### Action Extension

The Action Extension supports the opposite direction of integration: bringing context into SkillSwap from another app. A user may find text or a web page that gives them the idea for a request. The extension can receive shared content and help turn it into a draft instead of forcing the user to memorise or retype the information. Shared

extension data is passed through the App Group container so the extension and main app do not depend on direct in-process communication.

## Why Core Data?

Core Data was chosen because the main SkillSwap data is structured, relational and should remain available between launches even when there is no network connection. The persistence model contains related request and offer entities, with one request able to have multiple offers. This relationship is meaningful to the domain because offers only make sense in the context of a particular request.

The app does not allow Views or ViewModels to call Core Data directly. Instead, SkillSwapRepository defines the operations the rest of the app can use, while CoreDataSkillSwapRepository contains the persistence implementation. This keeps business rules out of the database layer and makes the repository replaceable with TestSkillSwapRepository during unit testing.

A local Core Data store also fits the scope of the prototype better than making network synchronisation a dependency for every interaction. Extension data is deliberately handled separately: the widget receives only the compact information it needs through the shared App Group container rather than trying to open the application's Core Data stack from another process.

## Architecture and domain rules

The application follows the flow Views -> ViewModels -> Use Cases -> Repository -> Core Data. The Use Case layer is where domain rules are enforced. Examples include requiring both sides of a request, preventing a member from offering on their own request, preventing new offers on a closed request, and ensuring only one offer is accepted. Typed domain errors are returned in language that can be shown to the user rather than exposing low-level persistence errors.

# Section 3 - Architecture Diagram

Insert the final one-page SkillSwap Human-System Architecture Diagram in the space below. The diagram should show the human-system boundary, the main application layers, Core Data, App Group communication, and each extension.

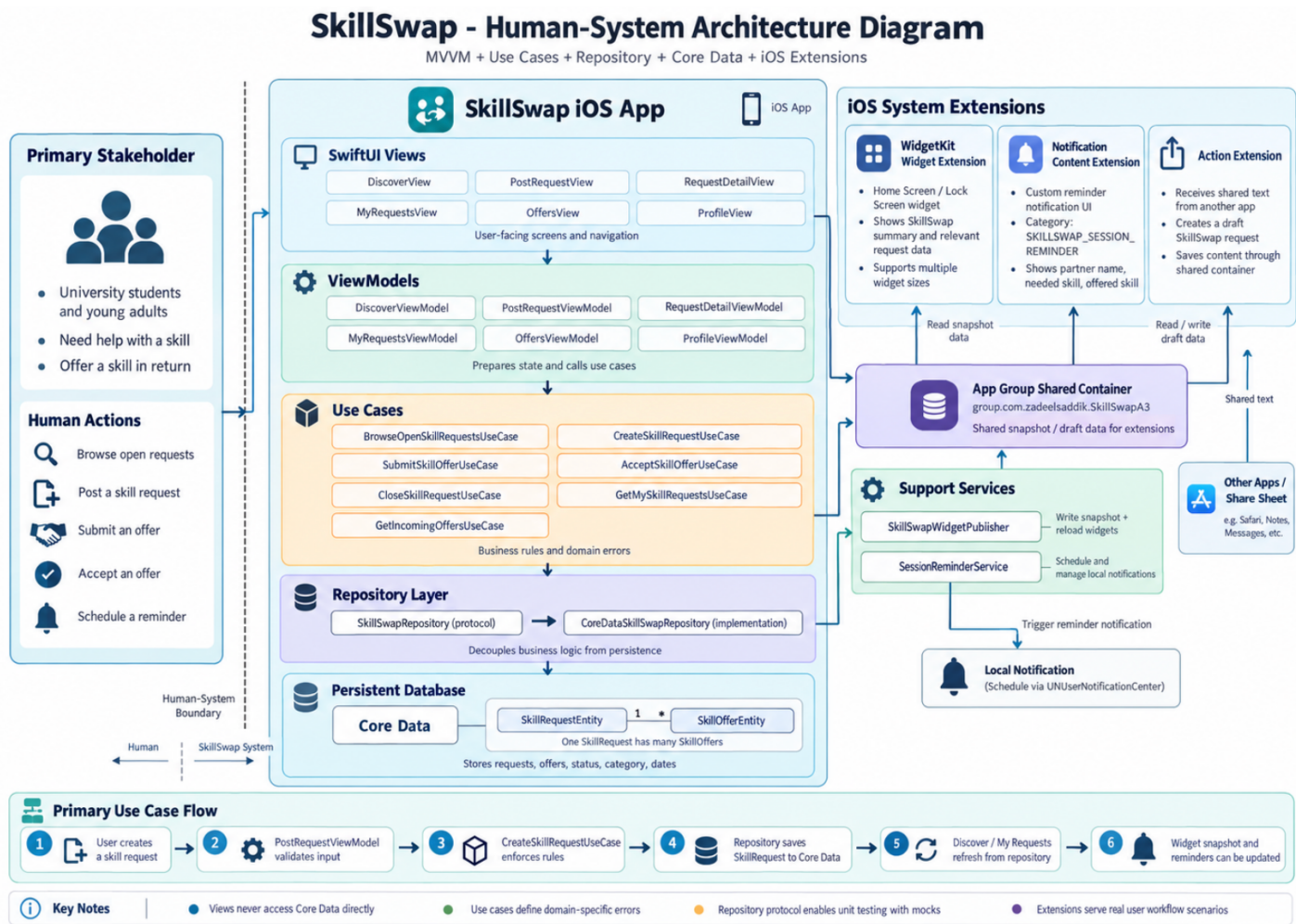


Figure 1. SkillSwap Human-System Architecture Diagram

## Section 4 - Reflective Report

*Reflective Report word count: approximately 830 words*

### Problem and justification

I chose SkillSwap because the problem felt small enough to be believable but still large enough to justify a proper system. Students regularly need things that sit in the gap between asking a friend and paying a professional: help with a subject, basic technology support, design work, language practice or a simple repair. At first I was mainly thinking about the money side. The more I worked on the app, the more I realised that the bigger design problem was coordination. Even if two people are willing to help each other, an exchange becomes messy when the need, the offered skill, the status and the offers are all spread across messages. The Department of Education material on student cost-of-living pressure gave me a stronger reason for focusing on students, but I tried not to claim that SkillSwap solves financial hardship. It only gives users another option for small exchanges that may otherwise be inconvenient or unaffordable.

### Extension design decisions

The extensions made me think about where SkillSwap should exist outside the main app. The widget is the clearest example. A user should not have to open the app just to see whether they have an active swap or pending activity, so the widget exposes a small snapshot instead of duplicating the whole interface. I think a regular user would notice if it disappeared because it removes a quick-glance part of the workflow. The notification content extension matters later in the exchange. Once two people have matched, a generic notification is not very useful, so the custom notification shows the partner and the two skills involved. The Action Extension serves a different moment: when the idea for a request starts in another app. It allows shared text to become part of a draft rather than making the user manually move information between apps.

### Database design decisions

I used Core Data because the core data is relational and local. SkillRequest and SkillOffer are separate domain concepts, and one request can have multiple offers. That relationship affected more than storage: accepting one offer changes the state of the request and changes what should happen to competing offers. The repository layer was important because I did not want those persistence details leaking into ViewModels. SkillSwapRepository defines what the business layer needs, CoreDataSkillSwapRepository handles the real store, and the tests can use a mock repository. That separation also made the code easier to reason about when I was debugging because I could tell whether a problem belonged to the business rules, presentation layer or persistence layer.

### Architecture under pressure

The biggest architectural pressure came from the fact that iOS extensions run separately from the main app. It would have been convenient to let the widget directly read whatever the app was using, but that would have coupled the extension to the persistence implementation and made the boundary less clear. I used the App Group as a deliberately small bridge instead. The main app publishes only the snapshot the widget needs and then asks WidgetKit to reload. This created extra configuration work, especially around entitlements, target membership and matching App Group identifiers, but it kept the data flow explicit. The same lesson applied to the other extensions: system integration is not just another screen inside the app, so the architecture has to respect process and capability boundaries.

## AI use declaration

I used AI as a development tool throughout this project. The base code for the SkillSwap application was generated with AI assistance. I did not treat that output as finished or automatically correct. I proof-checked the code, integrated it into the Xcode project and Git branches, built and ran the application, and troubleshooted the issues that appeared. Some of the AI-generated setup needed real correction, including duplicate widget entry points, incorrect target membership, missing Info.plist and entitlement references, App Group mismatches and merge problems in Git. I used AI again to help explain compiler messages, compare the implementation against the assessment criteria and draft documentation, but I tested the changes myself and decided which fixes to keep. This process made the limitations of generated code very obvious: producing code quickly was useful, but the more important part was understanding enough of the architecture and Xcode project structure to recognise when the generated result was wrong and repair it.

## What I would improve next

If I had another two weeks, I would focus on turning SkillSwap from a local prototype into a genuinely multi-user exchange system. The current architecture is suitable for demonstrating the domain rules and iOS integrations, but real users would need authenticated identities, remote synchronisation, moderation, trust and safety features, and a clearer way to arrange the practical details of a matched exchange. I would keep the Use Case and Repository boundaries and replace or extend the persistence implementation rather than moving network code into the Views. That would let the app grow without throwing away the separation that made the current version testable and understandable.

## References

- Tomaszewski, W., Johnstone, M., Robinson, M., Curry, M. & Kubler, M. (2023). Final Report: Targeted Review of Student Equity in Higher Education Programs and System Level Policy Levers. Report submitted to the Australian Government Department of Education. <https://www.education.gov.au/download/18228/accord-report-targeted-review-student-equity-higher-education-programs-and-system-level-policy/37667/document/pdf>
- Yuan, C.W.T., Hanrahan, B.V. & Carroll, J.M. (2018). 'Is there social capital in service exchange tools? Investigating timebanking use and social capital development'. Computers in Human Behavior, 81, 274-281. DOI: 10.1016/j.chb.2017.12.029. <https://doi.org/10.1016/j.chb.2017.12.029>